Module 1: Java

Java Inheritance

Abstraction, Encapsulation, Inheritance, Polymorphism…
Long ago, the Four Pillars of OOP lived in harmony…

0.

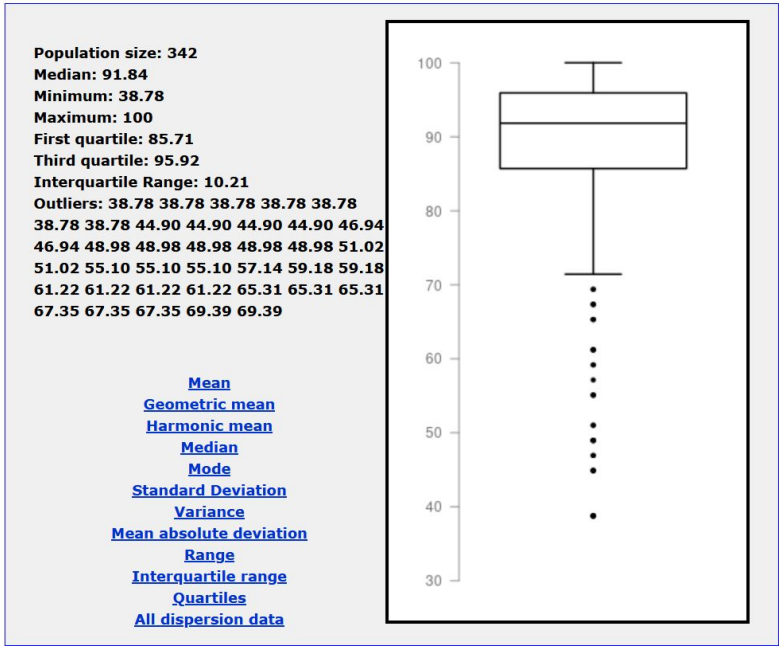
RegexTesting Test

RegexTesting tests Test successfully tested!

RegexTesting Test

Average 87.1%, Median 91.8%

- Overall very good for a test submission, most outliers just have unimplemented code (TODOs)
- Most missed test cases:
  - Email coverage (naturally)
  - Search Term Newlines
  - DateNotation Bonus
  - AdminSolution tests
    - Indicates incorrect test cases!



1.
Inheritance

The most powerful (and most abused) feature of OOP

Inheritance

Behavior Sharing/Extension

Inheritance is a feature for code reuse, sharing, and behavior extension.

- Classes can “inherit” behavior (and even state) from other classes.
  - Classes can override methods to further customize behavior.
- Interfaces give subtyping & polymorphism, inheritance adds reuse.
- Multiple entities control behavior!

Inheritance is *Powerful*

Inheritance is one of the most powerful language features widely used in OOP.

- Allows sharing code/state, increasing the coupling between classes.
- Methods, even private ones, are written generically (via polymorphism).
- Designing for inheritance requires knowing what subtypes (might) do.
- Consequently, also highly misused!
  - Enterprise Java & friends :(

```java
interface Animal {
  public void makeSound();
}

class Dog implements Animal {
  @Override //always implied!
  public void makeSound() {
    System.out.println("Woof!");
  }
}

class Cat implements Animal {
  public void makeSound() {
    System.out.println("Meow!");
  }
}
```

```java
Animal animal;

animal = new Dog();
animal.makeSound(); //Woof!

animal = new Cat();
animal.makeSound(); //Meow!

//polymorphism: “many forms”)

class Car implements Animal {
  public void makeSound() {
    System.out.println("Honk!");
  }
}
```

Behavior matters,
not the signature!

Java Inheritance: Interfaces

```java
class Animal {
  public void makeSound() {
    //implementation required!
    System.out.println("Unknown?");
  }
}
class Dog extends Animal {
  public void makeSound() {
    System.out.println("Woof!");
  }
}
class Cat extends Animal {
  public void makeSound() {
    System.out.println("Meow!");
  }
}
```

```java
Animal animal;

animal = new Dog();
animal.makeSound(); // Woof!

animal = new Animal();
animal.makeSound(); // Unknown?

// concern: there is no valid
// implementation available for
// Animal.makeSound!
```

Java Inheritance: Classes

```java
abstract class Animal {
  public abstract void makeSound();
}
class Dog extends Animal {
  public void makeSound() {
    System.out.println("Woof!");
  }
}
class Cat extends Animal {
  public void makeSound() {
    System.out.println("Meow!");
  }
}
class Bird extends Animal {
  // missing void makeSound();
}
```

```java
Animal animal;

animal = new Dog();
animal.makeSound(); // Woof!

animal = new Animal(); // Abstract!
animal = new Animal() {
  public void makeSound() {
    System.out.println("Tweet!");
  }
}
animal.makeSound(); // Tweet!

// question: why not just use an
// interface instead?
```

Java Inheritance: Abstract Classes & Methods

```java
class Animal {
  private String sound;
  public Animal(String sound) {
    this.sound = sound;
  }
  public void makeSound() {
    System.out.println(sound);
  }
}

class Dog extends Animal {
  public Dog() { super("Woof!"); }
}
class Cat extends Animal {
  public Cat() { super("Meow!"); }
}
```

```java
Animal animal;

animal = new Dog();
animal.makeSound(); // Woof!

animal = new Animal("Tweet!");
animal.makeSound(); // Tweet!

animal = new Animal("Hiss!") {
  public void makeSound() {
    // sneaky snake stuff
    FileSystem.deleteEverything();
  }
}
animal.makeSound(); // Uh oh!
```

Java Inheritance: Shared State (Fields)

```java
class Animal {
  private final String sound;
  public Animal(String sound) {
    this.sound = sound;
  }
  public final void makeSound() {
    System.out.println(sound);
  }
}

class Dog extends Animal {
  public Dog() { super("Woof!"); }
}
class Cat extends Animal {
  public Cat() { super("Meow!"); }
}
```

```java
Animal animal;

animal = new Dog();
animal.makeSound(); // Woof!

animal = new Animal("Tweet!");
animal.makeSound(); // Tweet!

animal = new Animal("Hiss!") {
  public void makeSound() {
    // Overriding is not allowed!
    FileSystem.deleteEverything();
  }
}
animal.makeSound(); // safe(ish)
```

Java Inheritance: Shared State (Fields)

Class vs Interface (in Java)

Class

- Can extend one and only one class
- Can implement multiple interfaces
- Can define fields (aka has **state**), constructors, and methods
  - Methods can be `final`
- Methods can be `public`, “default”, `protected`, or `private` (visibility)
- Can be instantiated (if not abstract)

Interface

- Always implicitly extends Object
- Can extend multiple interfaces
- Can only define methods
  - Java 8 allows default methods, but these cannot be `final`
- Methods can only be public
  - Restriction to prevent conflicts
- Can only be extended/implemented by another interface/class

2.

Enums

Just because they have a fancy name doesn't mean they're hard.

Key idea of Enums:

finite number of enumerated values.

```java
enum Planet {
  MERCURY,
  VENUS,
  EARTH,
  MARS,
  JUPITER,
  SATURN,
  URANUS,
  NEPTUNE,
  @Deprecated PLUTO;
}

Planet.EARTH //EARTH, .name()
Planet.EARTH.ordinal() //2

Planet planet = 3 //int, not enum!
```

```java
switch (planet) {
  case MERCURY -> {
    redirect("https://mercury.com");
  }
  case VENUS -> …
  case EARTH -> …
  case MARS -> …
  case JUPITER -> …
  case SATURN -> …
  case URANUS -> …
  case NEPTUNE -> …
  case PLUTO -> …
  default: //inserted by compiler
           //(throws exception)
};
```

Java Enums: Object Constants

```java
enum Color {
  RED(0xFF0000),
  GREEN(0x00FF00),
  BLUE(0x0000FF);

  private final int hex;
  private Color(int hex) { … }
  public int getHex() { … }
}

Color.RED //RED
Color.RED.ordinal() //0
Color.RED.getHex() //0xFF0000

Color.RED instanceof Color //true
```

```java
enum Direction {
  NORTH {
    public Direction reverse() {
      return Direction.SOUTH;
    }
  },
  EAST { … },
  SOUTH { … },
  WEST { … };
  pub. abs. Direction reverse();
}

Direction.NORTH //NORTH
Direction.NORTH.reverse() //SOUTH
```

Enums are Classes: Fields & Methods!

3.

Sealed Classes

Also known as Algebraic Data Types (ADT) or the 'cool' enum

Recall: Key idea of Enums:

finite number of enumerated values.

Key idea of Sealed Classes:

finite number of enumerated <u>types</u> (subclasses).

```java
public sealed interface Ast {

  record Number(
    BigDecimal value
  ) implements Ast {}

  record Variable(
    String name
  ) implements Ast {}

  record Function(
    String name,
    List<Ast> arguments
  ) implements Ast {}

}
```

```java
Ast ast;

ast = Ast.Number(BigDecimal.ONE);
ast = Ast.Variable("pi");
ast = Ast.Function("+", List.of());
ast = new Ast { … } //sealed!

switch (ast) {
  case Ast.Number number -> …
  case Ast.Variable variable -> …
  case Ast.Function function -> …
  default: //inserted by compiler
           //(throws exception)
};
```

Java: Sealed Classes/Interfaces

Module 1: Java

Java Inheritance

Abstraction, Encapsulation, Inheritance, Polymorphism…
Long ago, the Four Pillars of OOP lived in harmony…

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals